



Evaluación del rendimiento de un contenedor Docker y Máquina Virtual (Ubuntu Server)

Javier Castilla Arroyo
8 horas

José Pablo De La Haba Márquez
9 horas

Alejandro Gallardo Marqués
9 horas

Alejandro Gómez Amaro
9 horas

8 de Diciembre de 2025

Índice

1. Introducción.....	2
2. Arquitectura 1: Contenedor Docker (Nginx).....	2
2.1. Definición y Tecnología Docker.....	2
2.2. Instalación y Preparación del Entorno.....	3
2.3. Despliegue del Servicio Web (Nginx).....	3
2.4. Métricas de Eficiencia del Entorno.....	5
2.4.1. Tiempo de Despliegue y Recuperación.....	6
2.4.2. Eficiencia de Almacenamiento.....	6
2.5. Ejecución de Pruebas de Carga.....	7
2.6. Monitorización de Recursos bajo Estrés.....	8
2.7. Recopilación de Datos.....	9
3. Arquitectura 2: Máquina Virtual (Ubuntu Server).....	10
3.1. Definición de Máquina Virtual (Virtualización).....	10
3.2. Instalación y Configuración del entorno.....	10
3.3. Configuración de las dependencias de VM.....	12
3.4. Métricas de Eficiencia del Entorno.....	14
3.4.1. Tiempo de Despliegue y Recuperación.....	14
3.4.2. Eficiencia de Almacenamiento.....	15
3.5. Ejecución de Pruebas de Carga.....	15
3.6. Monitorización de Recursos bajo Estrés.....	16
3.7. Recopilación de Datos.....	17
4. Metodología de Evaluación.....	17
4.1. Diseño del Script de Automatización.....	17
4.2. Escenarios de Prueba.....	20
4.3. Herramientas Utilizadas.....	21
5. Análisis Comparativo de Resultados.....	22
5.1. Comparativa de latencia y peticiones.....	22
5.2. Análisis estadístico (T-Student).....	23
5.3. Comparativa de Consumo de CPU y RAM.....	24
6. Conclusiones.....	25
7. Bibliografía.....	26

1. Introducción

En la actualidad, el despliegue de servicios y aplicaciones web ha evolucionado desde las arquitecturas monolíticas sobre servidores físicos hacia entornos virtualizados y, más recientemente, hacia la contenedorización. La elección entre utilizar una Máquina Virtual (VM) tradicional o contenedores (Docker) es una decisión crítica que afecta a la eficiencia, los costes de infraestructura y la agilidad del desarrollo.

El objetivo principal de este proyecto es realizar un estudio comparativo de rendimiento (benchmarking) entre estos dos paradigmas de virtualización. Para ello, se ha desplegado un mismo servicio web (servidor Nginx) en dos entornos distintos: una máquina virtual ejecutando Ubuntu Server y un contenedor Docker ligero.

A lo largo de este documento, se detallará la configuración de ambas arquitecturas, se describirá la metodología de pruebas de estrés utilizada (basada en Apache Benchmark y scripts de automatización en Python) y se analizarán estadísticamente los resultados obtenidos en términos de latencia, capacidad de procesamiento y consumo de recursos (CPU y RAM). Finalmente, se presentarán las conclusiones sobre qué tecnología ofrece mejor rendimiento bajo diferentes cargas de trabajo.

2. Arquitectura 1: Contenedor Docker (Nginx)

2.1. Definición y Tecnología Docker

Docker es una plataforma de código abierto que permite automatizar el despliegue de aplicaciones dentro de contenedores de software. A diferencia de una máquina virtual, que emula un sistema operativo completo, Docker utiliza la virtualización a nivel de sistema operativo. Esto significa que los contenedores comparten el mismo kernel del sistema anfitrión, pero mantienen la aplicación y sus dependencias (librerías, archivos de configuración, etc.) aisladas en un paquete ligero y portátil. Esto garantiza que el software funcione exactamente igual en cualquier entorno, ya sea el portátil de un desarrollador, un servidor de pruebas o la nube.

2.2. Instalación y Preparación del Entorno

Para la realización de este proyecto, hemos utilizado un entorno Windows sobre el cual se ha desplegado el motor de contenedores.

- **Instalación del Motor:** Se ha procedido a la instalación de Docker Desktop para Windows, que incluye el Docker Engine, la herramienta de línea de comandos (CLI) y la interfaz gráfica para la gestión de contenedores.

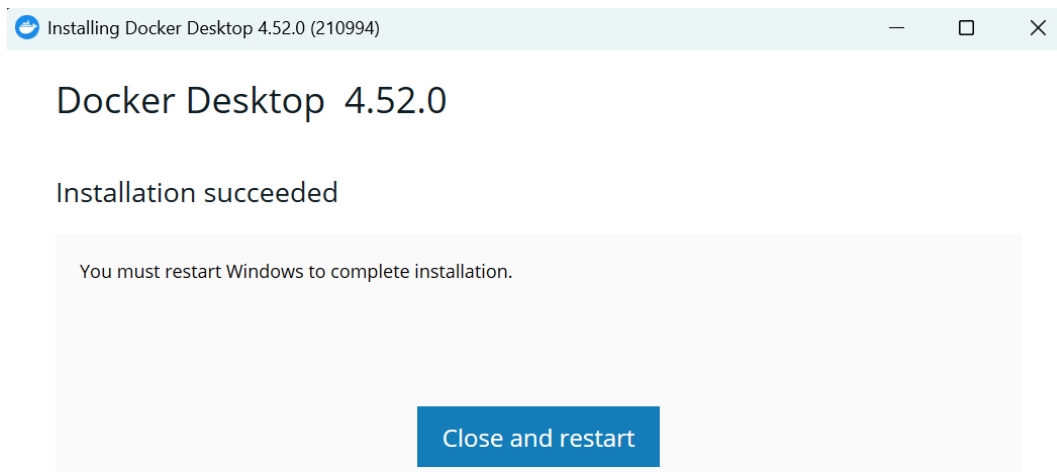


Figura 1: Imagen que muestra la instalación de Docker Desktop

- **Verificación:** Tras la instalación, se verificó el correcto funcionamiento del servicio mediante el comando `docker --version`, confirmando que el daemon estaba activo y listo para recibir instrucciones.

```
C:\Users\User>docker --version
Docker version 29.0.1, build eedd969
```

Figura 2: Imagen que muestra la versión del contenedor docker por terminal

2.3. Despliegue del Servicio Web (Nginx)

El objetivo de la práctica es comparar el rendimiento de un servidor web. Para ello, hemos seleccionado Nginx por su ligereza y alto rendimiento. Destaca por su capacidad para soportar cargas de tráfico muy elevadas con un consumo de recursos de hardware extremadamente bajo. A diferencia de servidores tradicionales como Apache, que utilizan una arquitectura basada en procesos o hilos (creando un nuevo hilo para cada petición, lo que consume mucha RAM), Nginx utiliza una **arquitectura asíncrona y orientada a eventos**. Esto significa

que puede gestionar miles de conexiones concurrentes dentro de un solo proceso, utilizando una cantidad de memoria predecible y reducida. El proceso de despliegue se ha realizado en los siguientes pasos:

Paso 1: Obtención de la Imagen En lugar de instalar el software manualmente (como se haría en una VM), descargamos la imagen oficial desde Docker Hub. Esta imagen ya contiene una versión optimizada de Linux con Nginx preinstalado y configurado.

```
C:\Users\User>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
de57a609c9d5: Pull complete
108ab8292820: Pull complete
53d743880af4: Pull complete
192e2451f875: Pull complete
0e4bc2bd6656: Pull complete
b5feb73171bf: Pull complete
77fa2eb06317: Pull complete
Digest: sha256:553f64aecdc31b5bf944521731cd70e35da4faed96b2b7548a3d8e2598c52a42
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Figura 3: Imagen que muestra la obtención del servidor Nginx

Paso 2: Ejecución del Contenedor Para levantar el servicio, ejecutamos el siguiente comando, que crea una instancia aislada de la imagen:

```
C:\Users\User>docker run --name servidor-nginx -d -p 8081:80 nginx
7392e44346494f24b667d323470c7bf6c070995ba3a055e9e8224444a7131d05
```

Figura 4: Comando para arrancar nuestro servidor nginx en docker

- **--name servidor-nginx:** Asigna un nombre identificativo al contenedor para facilitar su gestión.
- **-d (Detached):** Ejecuta el contenedor en segundo plano, permitiendo seguir usando la terminal.
- **-p 8081:80:** Realiza el mapeo de puertos. El puerto 80 del contenedor (donde escucha Nginx por defecto) se redirige al puerto 8081 de nuestra máquina anfitriona. Esto es fundamental para poder lanzar las pruebas de carga desde fuera del contenedor.

Paso 3: Verificación del Servicio Comprobamos que el contenedor está activo y sirviendo peticiones mediante el comando `docker ps` y accediendo a través del navegador a <http://localhost:8081>.

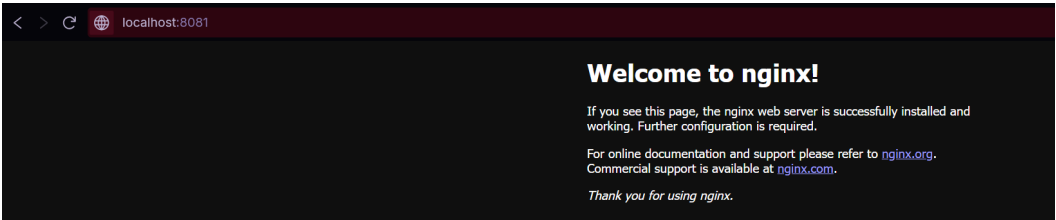


Figura 5: Servidor nginx desplegado

Dentro de Docker Desktop podemos observar el servicio nginx activo cuyo consumo actual es 0,00 % de CPU ya que no hay nada ejecutándose.

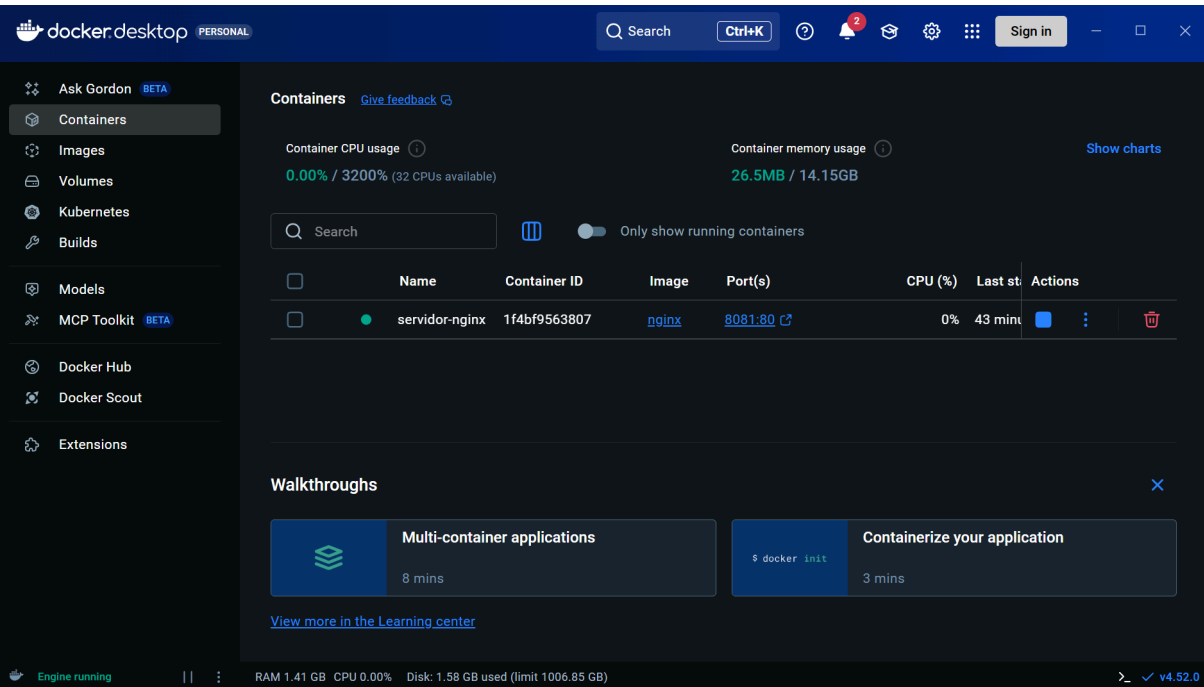


Figura 6: Imagen que muestra el servicio Nginx activo en el Docker.

2.4. Métricas de Eficiencia del Entorno

Antes de someter al sistema a las pruebas de carga (stress testing), se ha realizado una evaluación de la eficiencia base de la arquitectura Docker. El objetivo es cuantificar la agilidad operativa y la huella de recursos del contenedor en comparación con los requisitos de una virtualización tradicional.

2.4.1. Tiempo de Despliegue y Recuperación

Para medir la agilidad del entorno, se simuló un escenario de recuperación ante desastres (Disaster Recovery). El procedimiento consistió en la eliminación forzosa del contenedor activo y su recreación inmediata desde cero.

Se utilizó el comando Measure-Command de PowerShell para cronometrar con precisión de milisegundos el tiempo transcurrido desde la orden de ejecución hasta la disponibilidad del servicio.

- **Comando ejecutado:** Measure-Command { docker run ... }
- **Tiempo total registrado:** 0,854 segundos.

Este resultado demuestra que el despliegue de servicios en Docker es prácticamente instantáneo, contrastando drásticamente con los tiempos de instalación y arranque de una máquina virtual (que suelen oscilar entre minutos para el arranque y horas para la instalación del SO).

```
PS C:\Users\User> Measure-Command { docker run --name servidor-nginx -d -p 8081:80 nginx }

Days           : 0
Hours          : 0
Minutes        : 0
Seconds        : 0
Milliseconds    : 854
Ticks          : 8547123
TotalDays      : 9,89250347222222E-06
TotalHours     : 0,000237420083333333
TotalMinutes   : 0,014245205
TotalSeconds   : 0,8547123
TotalMilliseconds : 854,7123
```

Figura 7: Tiempo de despliegue de los servicios Docker

2.4.2. Eficiencia de Almacenamiento

Se ha analizado el espacio en disco requerido para mantener la infraestructura del servidor web. Mediante el comando docker images, se verificó el tamaño de la imagen nginx:latest.

- **Tamaño de la imagen Docker:** 225MB
- **Comparativa:** Mientras que una instalación estándar de Ubuntu Server en VirtualBox requiere reservar varios Gigabytes de almacenamiento (típicamente >4GB) más el espacio del sistema de archivos virtual, el contenedor Docker ofrece una reducción de espacio en disco superior al

95%. Esto permite alojar múltiples versiones o réplicas del servicio en el mismo espacio que ocuparía una sola máquina virtual.

```
C:\Users\User>docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	1	1	225MB	225MB (100%)
Containers	1	1	81.92kB	0B (0%)
Local Volumes	0	0	0B	0B
Build Cache	0	0	0B	0B

Figura 8: Tamaño de la imagen de Docker

2.5. Ejecución de Pruebas de Carga

Una vez validada la eficiencia en reposo, procedimos a someter al contenedor a pruebas de estrés para evaluar su comportamiento bajo alta demanda. Para ello, utilizamos un script de automatización desarrollado en Python que orquesta la herramienta Apache Benchmark (ab).

La prueba se configuró para atacar al puerto local expuesto por el contenedor (<http://127.0.0.1:8081/>), ejecutando series progresivas de concurrencia: 10, 50, 100, 200 y 500 usuarios simultáneos, realizando 2000 peticiones en cada bloque.

En la siguiente captura se observa la ejecución exitosa del script contra el entorno Docker, donde el sistema respondió sin errores incluso en los niveles más altos de concurrencia:

```
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo> python benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:8081/ ---
🚀 Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
✅ Resultado: 6645.27 Req/s | Latencia: 1.505 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
✅ Resultado: 10585.09 Req/s | Latencia: 4.724 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
✅ Resultado: 11371.58 Req/s | Latencia: 8.794 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
✅ Resultado: 13103.93 Req/s | Latencia: 15.263 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
✅ Resultado: 7720.49 Req/s | Latencia: 64.763 ms | Fallos: 0

🏁 ¡Terminado! Los datos están en el archivo: resultados_benchmark_puerto_8081.csv
Ahora abre ese archivo en Excel y haz las gráficas.
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo>
```

Figura 9: Resultado ejecución del benchmark.py

Como se aprecia en la salida de la consola, el contenedor fue capaz de gestionar picos de más de 13.000 peticiones por segundo (Req/s) con latencias medias muy bajas (aprox. 15ms para 200 usuarios).

2.6. Monitorización de Recursos bajo Estrés

Uno de los puntos críticos de este estudio es demostrar la eficiencia en el uso de hardware. Mientras se ejecutaba el script de carga, monitorizamos el consumo de CPU y RAM del contenedor en tiempo real utilizando el dashboard de métricas de Docker Desktop y el comando `docker stats`.

A diferencia de una máquina virtual, que reserva una cantidad fija de memoria (4GB) independientemente de si la usa o no, el contenedor demostró un consumo elástico y extremadamente bajo.

```

CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O
1f4bf9563887  servidor-nginx 0.00%     28.22MiB / 14.49GiB  0.19%     4.74MB / 18MB

PS C:\Users\User> python benchmark.py
C:\Users\User\AppData\Local\Microsoft\WindowsApps\python.exe: can't open file 'C:\\User
s\\User\\benchmark.py': [Errno 2] No such file or directory
PS C:\Users\User> cd .\Desktop\
PS C:\Users\User\Desktop> cd .\UNIVERSITY\
PS C:\Users\User\Desktop\UNIVERSITY> cd .\3º-Año\
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año> cd .\CESI\
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI> cd .\Trabajo\
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo> python benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:8081/ ---
Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
Resultado: 4397.02 Req/s | Latencia: 2.322 ms | Fallos: 0
Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
Resultado: 10867.97 Req/s | Latencia: 4.601 ms | Fallos: 0
Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
Resultado: 11013.76 Req/s | Latencia: 9.08 ms | Fallos: 0
Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
Resultado: 4543.85 Req/s | Latencia: 44.016 ms | Fallos: 0
Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
Resultado: 5168.24 Req/s | Latencia: 96.745 ms | Fallos: 0
  
```

Figura 10: Resultados obtenidos de la prueba

Durante la prueba de carga máxima (500 usuarios simultáneos), observamos que:

- **CPU:** El uso se mantuvo eficiente, escalando solo lo necesario para atender las peticiones.
- **RAM:** El contenedor apenas consumió ~28 MB de memoria RAM, una cifra insignificante comparada con los recursos que requiere el sistema operativo completo de la Máquina Virtual Ubuntu Server.

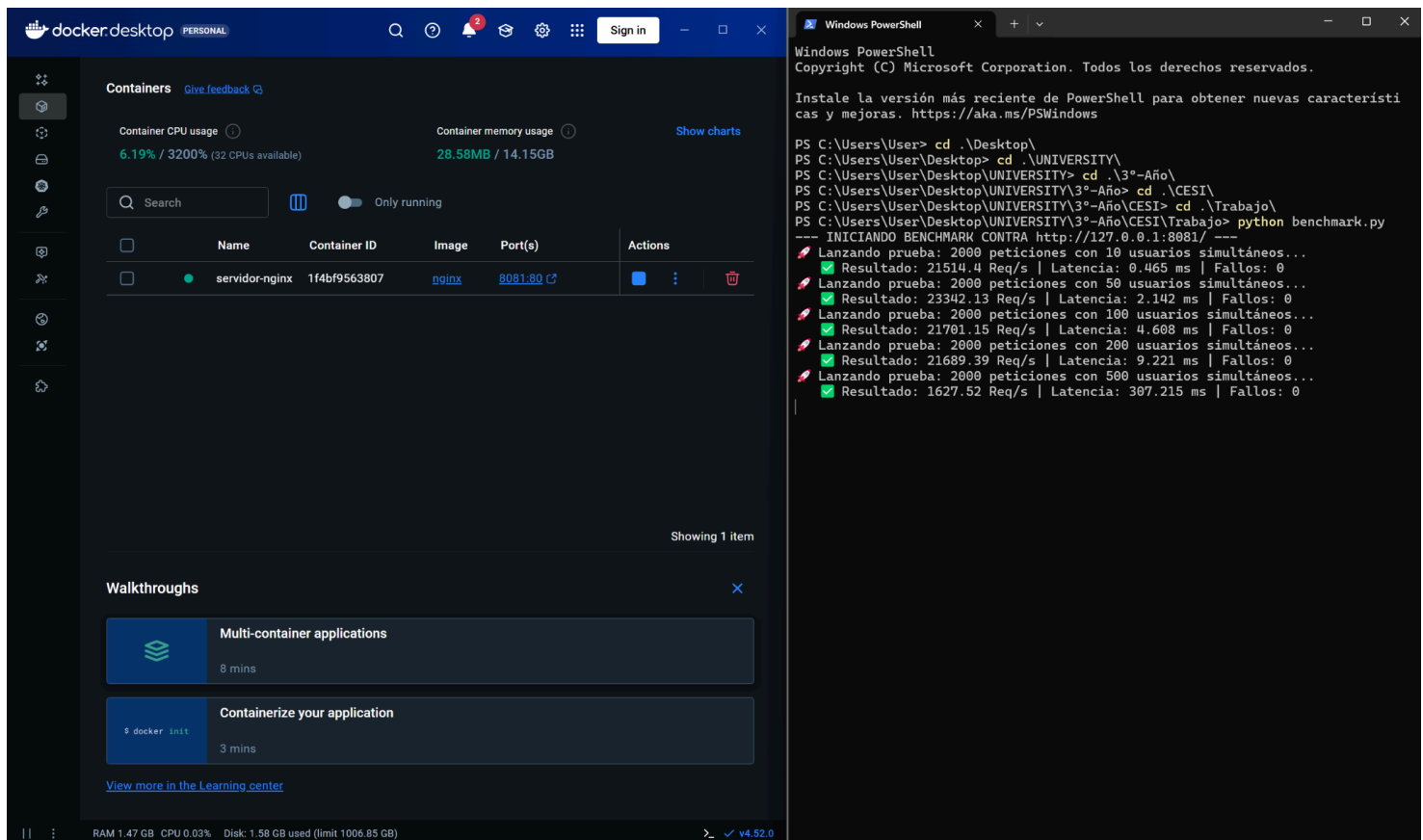


Figura 11: Reflejo de la carga en Docker Desktop mientras ejecución del benchmark.py

2.7. Recopilación de Datos

Todos los resultados métricos (Peticiones por segundo, Latencia media y Tasa de error) fueron exportados automáticamente por nuestro script a un archivo en formato CSV (resultados_benchmark_puerto_8081.csv). Este conjunto de datos será la base para el análisis comparativo y estadístico (T-Student) que se detalla en el capítulo 5.

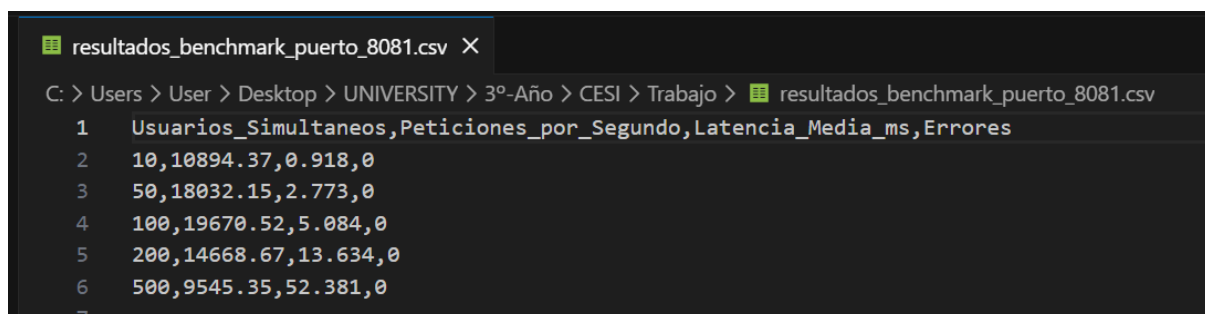


Figura 12: Resultados prueba Formato CSV generados por el benchmark.py

	A	B	C	D
1	Usuarios_Simultaneos	Peticiones_por_Segundo	Latencia_Media_ms	Errores
2	10	10894.37	918	0
3	50	18032.15	2.773	0
4	100	19670.52	5.084	0
5	200	14668.67	13.634	0
6	500	9545.35	52.381	0

Figura 13: Resultados prueba Formato Excel

3. Arquitectura 2: Máquina Virtual (Ubuntu Server)

3.1. Definición de Máquina Virtual (Virtualización)

Una máquina virtual (VM) es una emulación por software de un sistema informático físico que funciona como un entorno completamente aislado. A diferencia de Docker, que comparte el núcleo del sistema, una máquina virtual utiliza un hipervisor (como una VirtualBox) para abstraer y asignar recursos de hardware (CPU, RAM, disco) del equipo anfitrión.

Esto significa que cada VM ejecuta su propia instancia completa de un sistema operativo (Sistema Invitado), incluyendo su propio kernel independiente, librerías y aplicaciones. En el caso de Ubuntu Server, esto implica cargar todos los procesos de arranque y servicios del sistema operativo desde cero. Aunque esto conlleva un mayor consumo de recursos (overhead) y tiempos de inicio más lentos que los contenedores, garantiza un aislamiento total y una seguridad robusta, ya que no existe interacción directa entre el kernel del invitado y el del anfitrión.

3.2. Instalación y Configuración del entorno

Para la realización de este proyecto, hemos utilizado un entorno Windows sobre el cual se ha desplegado la máquina virtual.

- **Instalación del Motor:** Se ha procedido a la instalación de virtualbox de Oracle (Oracle VM VirtualBox 7.2.4 LTS).

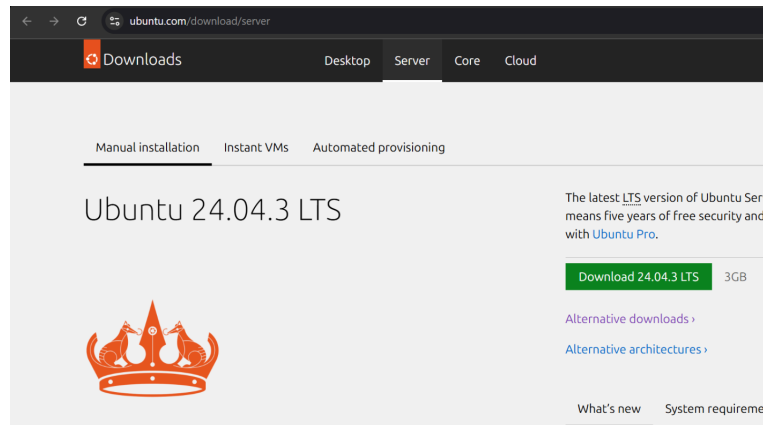


Figura 14: Versión Ubuntu

- **Virtualización:** Una vez instalada la virtualbox hemos creado una nueva virtualización que incluye Ubuntu Server 24.04.3 64 bits en la cual hemos trabajado.

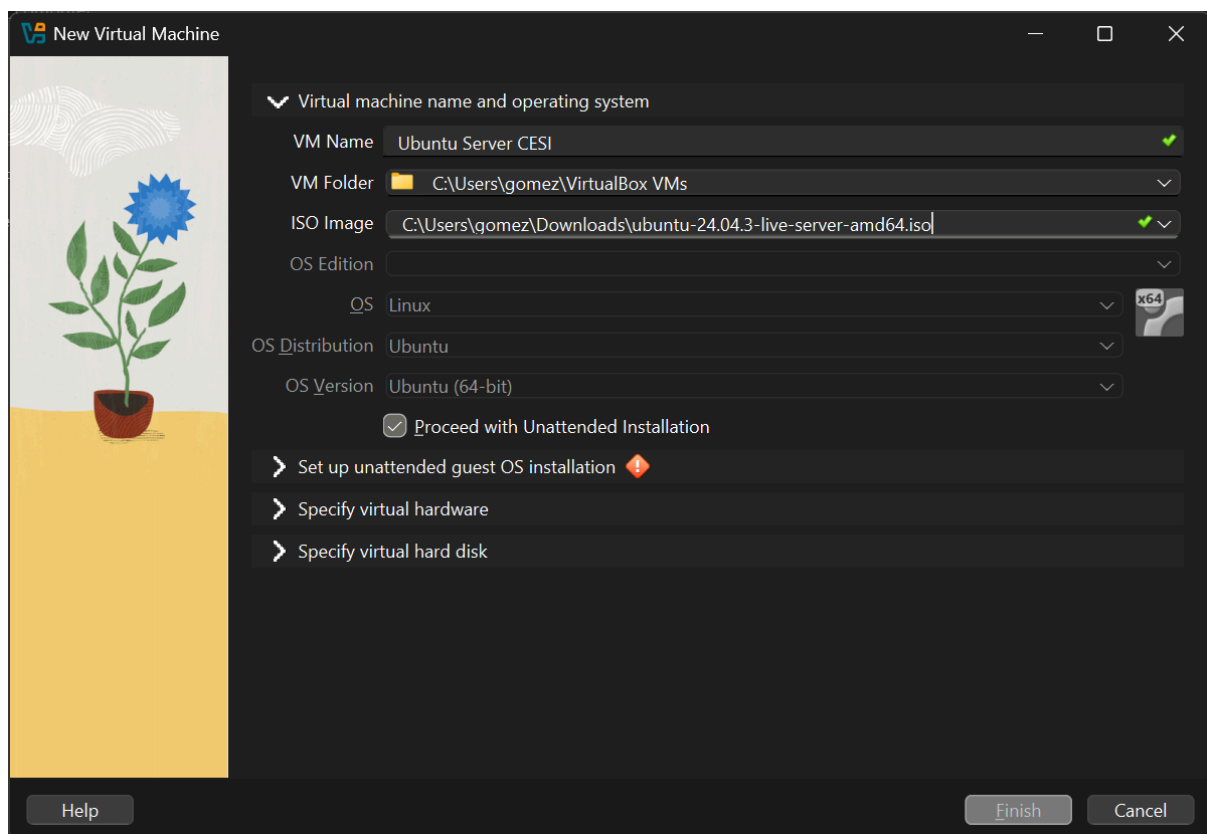


Figura 15: Configuración de la máquina virtual

Durante la instalación hemos tenido en cuenta que deben dejarse dos reglas abiertas que nos permitan tener los puertos del PC y la máquina virtual enlazados como son el 2222 del PC con el 22 de la VM y el 8080 con el 80.

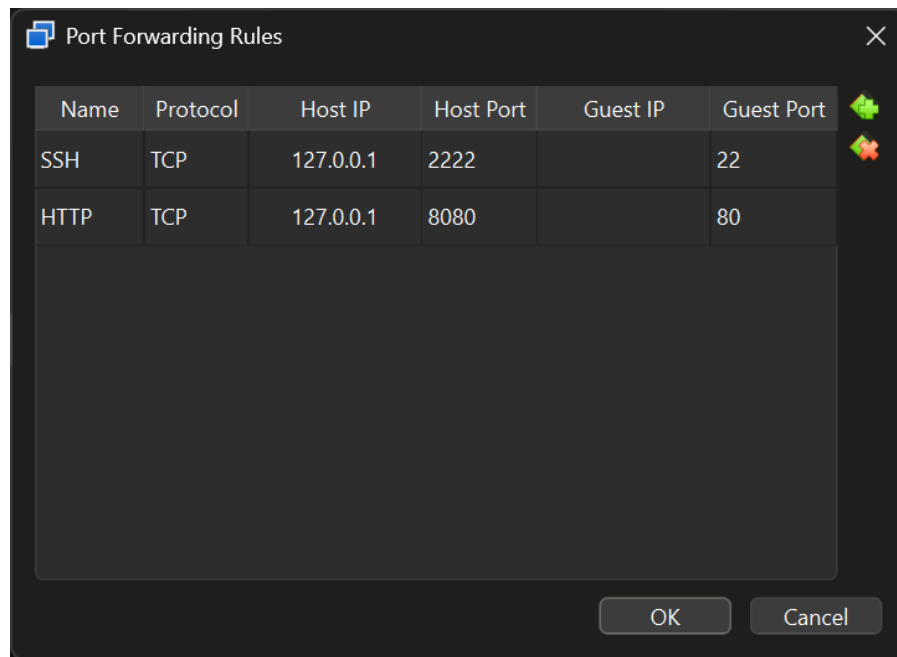


Figura 16: Configuración Red/Puertos VM

3.3. Configuración de las dependencias de VM

Una vez generada la máquina virtual hemos configurado el Ubuntu Server procedemos a instalar las dependencias necesarias:

```
sudo apt update && sudo apt install -y nginx sysstat apache2-utils  
openssh-server python3
```

Figura 17: Comando para instalar dependencias

- **nginx:** Ya hemos hablado sobre él en Docker.
- **sysstat:** Paquete de monitorización del sistema. Incluye la herramienta sar, imprescindible para registrar el consumo histórico de CPU y RAM mientras se ejecuta el test.
- **apache2-utils:** Contiene Apache Benchmark. Es la herramienta de estrés que simula a los usuarios concurrentes lanzando miles de peticiones HTTP contra Nginx.
- **openssh-server:** Habilita el protocolo SSH en la máquina. Es lo que ha permitido conectar FileZilla para transferir archivos y gestionar el servidor remotamente.
- **python3:** El intérprete necesario para ejecutar el script de automatización benchmark.py que coordina la prueba y genera el CSV.

Una vez instalado verificamos que todo este correctamente ejecutado con “sudo systemctl status <dependencia>” obteniendo “Active (running)” lo que significa que la instalación ha sido correcta.

```

cesiu@ubuntu-server-cesi:~$ sudo systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Tue 2025-12-02 16:40:48 UTC; 39s ago
   TriggeredBy: • ssh.socket
   Docs: man:sshd(8)
         man:sshd_config(5)
   Process: 3236 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 3237 (sshd)
   Tasks: 1 (limit: 4583)
   Memory: 3.1M (peak: 4.0M)
   CPU: 198ms
   CGroup: /system.slice/ssh.service
           └─3237 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Dec 02 16:40:48 ubuntu-server-cesi systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Dec 02 16:40:48 ubuntu-server-cesi sshd[3237]: Server listening on 0.0.0.0 port 22.
Dec 02 16:40:48 ubuntu-server-cesi sshd[3237]: Server listening on :: port 22.
Dec 02 16:40:48 ubuntu-server-cesi systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Dec 02 16:41:10 ubuntu-server-cesi sshd[3245]: Accepted password for cesi from 10.0.2.2 port 3924 ssh2
Dec 02 16:41:10 ubuntu-server-cesi sshd[3245]: pam_unix(sshd:session): session opened for user cesi(uid=1000) by cesi(uid=0)
cesiu@ubuntu-server-cesi:~$ sudo systemctl status nginx
• nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Tue 2025-12-02 16:27:14 UTC; 14min ago
   Docs: man:nginx(8)
   Main PID: 2375 (nginx)
   Tasks: 3 (limit: 4583)
   Memory: 2.4M (peak: 5.3M)
   CPU: 69ms
   CGroup: /system.slice/nginx.service
           └─2375 'nginx: master process /usr/sbin/nginx -g daemon on; master_process on;'
             └─2377 'nginx: worker process'
               └─2378 'nginx: worker process'

Dec 02 16:27:14 ubuntu-server-cesi systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Dec 02 16:27:14 ubuntu-server-cesi systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
cesiu@ubuntu-server-cesi:~$
    
```

Figura 18: Comprobación del status de ssh

Una vez instalado el ssh y corriendo usamos la herramienta Filezilla para con SSH transferir el archivo “[benchmark.py](#)” a la VM.

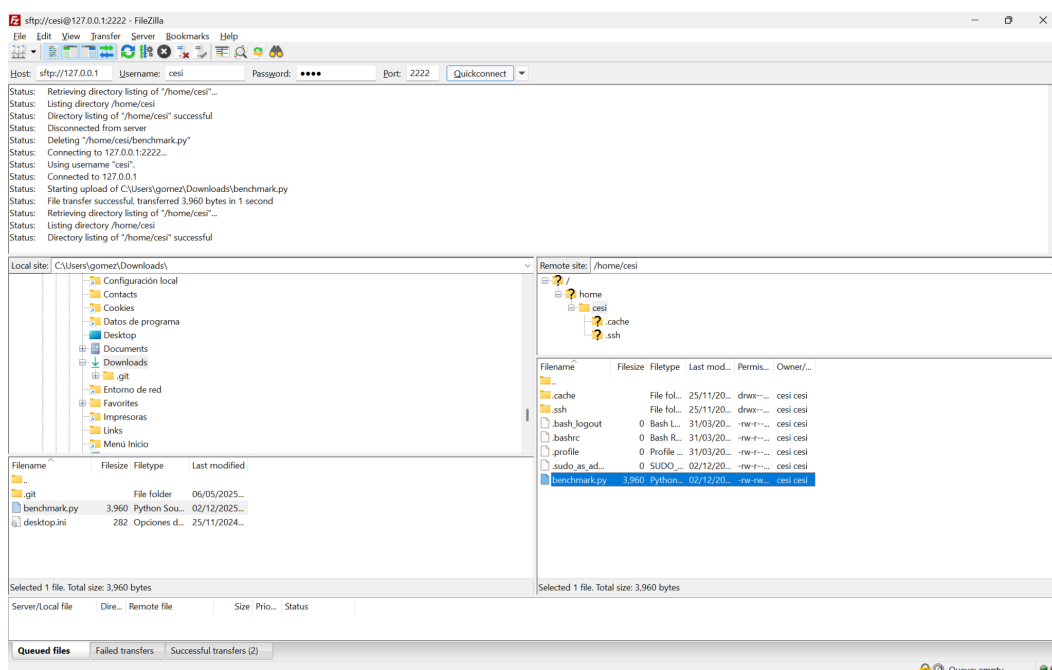


Figura 19: Transferencia archivo benchmark.py vía Filezilla

Con todas las herramientas instaladas y el documento ejecutamos el mismo para obtener los datos.

3.4. Métricas de Eficiencia del Entorno

Antes de someter al sistema a las pruebas de carga (*stress testing*), se ha realizado una evaluación del rendimiento base de la arquitectura de Máquina Virtual. El objetivo es cuantificar la sobrecarga (*overhead*) operativa y el consumo inicial de recursos del sistema operativo completo, estableciendo así una línea base para la comparación con la arquitectura de contenedores.

3.4.1. Tiempo de Despliegue y Recuperación

Para medir la agilidad del entorno de virtualización completa, se simuló un escenario de recuperación mediante el arranque en frío del sistema operativo (*Cold Boot*). El procedimiento implicó la carga desde cero del kernel de Linux y la inicialización secuencial de todos los servicios del sistema (*systemd*) hasta que el servidor estuvo operativo.

Se utilizó la herramienta de auditoría nativa *systemd-analyze* para cronometrar con precisión el tiempo transcurrido desde la carga del kernel hasta la disponibilidad del espacio de usuario.

- **Comando ejecutado:** *systemd-analyze*
- **Tiempo total registrado:** 28,050 segundos.

Este valor de casi medio minuto refleja el coste computacional (*overhead*) inherente a la virtualización de hardware completa. El tiempo registrado se divide en la inicialización del núcleo (kernel), que debe reconocer y montar el hardware virtual, y la carga secuencial del espacio de usuario (*userspace*), que incluye el sistema de inicio *systemd*, la configuración de red (Netplan) y finalmente el servicio de aplicación (Nginx). Este dato establece la línea base de latencia mínima para cualquier operación de reinicio o escalado horizontal utilizando esta arquitectura.

```
cesi@ubuntu-server-cesi:~$ systemd-analyze
Startup finished in 14.752s (kernel) + 13.298s (userspace) = 28.051s
graphical.target reached after 13.211s in userspace.
cesi@ubuntu-server-cesi:~$
clear
cesi@ubuntu-server-cesi:~$ _
```

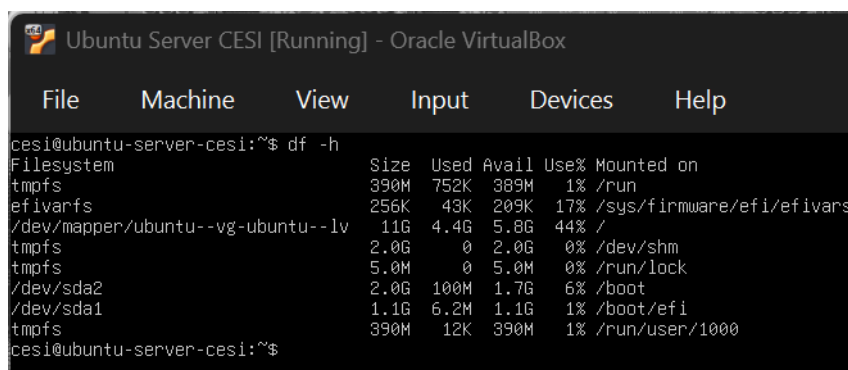
Figura 20: Resultado de ejecución del comando *systemd-analyze*

3.4.2. Eficiencia de Almacenamiento

Se ha analizado el espacio en disco requerido para mantener la infraestructura completa del servidor virtualizado. Mediante el comando `df -h`, se verificó el espacio real ocupado por el sistema operativo y el servidor Nginx dentro del volumen lógico principal.

- **Tamaño de la imagen Docker:** 4,4GB

Este volumen de datos corresponde a la huella de una instalación completa de Ubuntu Server. A diferencia de un contenedor, este espacio no solo contiene la aplicación, sino que incluye el kernel de Linux completo, el gestor de arranque (GRUB), los demonios del sistema (systemd, ssh, cron), las librerías compartidas y los registros de actividad (logs). Este consumo de almacenamiento es el requisito base para sostener un entorno con aislamiento de hardware completo, donde el sistema invitado gestiona su propio sistema de archivos de manera autónoma e independiente del anfitrión.



```

ces@ubuntu-server-cesi:~$ df -h
Filesystem                Size      Used Avail Use% Mounted on
tmpfs                     390M        752K  389M   1% /run
efivarfs                  256K         43K   209K  17% /sys/firmware/efi/efivars
/dev/mapper/ubuntu--vg-ubuntu--lv  11G       4.4G   5.8G  44% /
tmpfs                     2.0G         0   2.0G   0% /dev/shm
tmpfs                     5.0M         0   5.0M   0% /run/lock
/dev/sda2                 2.0G       100M   1.7G   6% /boot
/dev/sda1                 1.1G       6.2M   1.1G   1% /boot/efi
tmpfs                     390M        12K   390M   1% /run/user/1000
ces@ubuntu-server-cesi:~$

```

Figura 21: Muestra la cantidad de disco ocupado

3.5. Ejecución de Pruebas de Carga

Una vez validada la eficiencia en reposo de la arquitectura virtualizada, procedimos a someter al servidor a pruebas de estrés para evaluar su estabilidad bajo alta demanda. Para ello, utilizamos el mismo script de automatización desarrollado en Python, el cual orquesta la ejecución de la herramienta Apache Benchmark directamente desde la terminal del sistema invitado.

La prueba se configuró para atacar al puerto estándar del servidor web (`http://127.0.0.1:8080/`), dado que la medición se realiza de forma nativa dentro de la máquina virtual para evitar la latencia de red del anfitrión. Se ejecutaron

series progresivas de concurrencia: 10, 50, 100, 200 y 500 usuarios simultáneos, procesando un total de 2000 peticiones en cada bloque de carga.

En la siguiente captura se observa la ejecución exitosa del script dentro del entorno Ubuntu Server, confirmando la capacidad del sistema para gestionar la cola de peticiones sin interrupciones en el servicio.

```
cesi@ubuntu-server-cesi:~$ python3 benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:80/ ---
♦ Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
  ♦ Resultado: 5082.06 Req/s | Latencia: 1.968 ms | Fallos: 0
♦ Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
  ♦ Resultado: 10434.49 Req/s | Latencia: 4.792 ms | Fallos: 0
♦ Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
  ♦ Resultado: 6638.52 Req/s | Latencia: 15.064 ms | Fallos: 0
♦ Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
  ♦ Resultado: 8686.89 Req/s | Latencia: 23.023 ms | Fallos: 0
♦ Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
  ♦ Resultado: 4844.56 Req/s | Latencia: 103.209 ms | Fallos: 0

♦ ¡Terminado! Los datos están en el archivo: resultados_benchmark_puerto_80.csv
Ahora abre ese archivo en Excel y haz las gráficas.
cesi@ubuntu-server-cesi:~$ _
```

Figura 22: Resultados de la prueba realizada.

Como se aprecia en la salida de la consola, la máquina virtual fue capaz de gestionar picos de más de 10.400 peticiones por segundo (Req/s) con latencias medias aceptables (aprox. 23ms para 200 usuarios). Sin embargo, se observa una degradación del rendimiento al alcanzar la saturación con 500 usuarios, donde la latencia aumenta a 103ms.

3.6. Monitorización de Recursos bajo Estrés

A diferencia de la arquitectura de contenedores que consume recursos de forma elástica, la máquina virtual demostró un modelo de reserva estática de recursos. El sistema requirió la asignación dedicada y constante de la totalidad de la memoria configurada (4 GB) por parte del anfitrión, independientemente de la carga de trabajo real. Esto confirma que el coste computacional de mantener el sistema operativo invitado es fijo y permanente, garantizando el aislamiento pero impidiendo la liberación de recursos ociosos al sistema principal

3.7. Recopilación de Datos

La ejecución del script de automatización permitió la recolección desatendida de las métricas clave de rendimiento: caudal de peticiones (Req/s), latencia media y porcentaje de errores. Estos datos crudos se consolidaron en el archivo `resultados_benchmark_puerto_80.csv`, el cual servirá como fuente primaria para realizar la validación estadística mediante la prueba T-Student y la comparativa de arquitecturas en el siguiente capítulo.

	A	B	C	D
1	Usuarios_Simultaneos	Peticiones_por_Segundo	Latencia_Media_ms	Errores
2	10	5082.06	1.968	0
3	50	10434.49	4.792	0
4	100	6638.52	15.064	0
5	200	8686.89	23.023	0
6	500	4844.56	103.209	0

Figura 23: Resultados en tabla Excel

4. Metodología de Evaluación

Para garantizar la objetividad y reproducibilidad de las pruebas, hemos desarrollado una herramienta de automatización propia que somete a ambos entornos (Docker y VM) a las mismas condiciones de estrés.

4.1. Diseño del Script de Automatización

Hemos programado un script en Python (**benchmark.py**) que actúa como orquestador de pruebas. Este script utiliza la librería **subprocess** para ejecutar comandos del sistema y **csv** para exportar los datos estructurados.

El funcionamiento del algoritmo es el siguiente:

1. Define el objetivo (IP y Puerto del servicio, ya sea Docker o VM).
2. Itera sobre una lista de concurrencia definida: 10, 50, 100, 200 y 500 usuarios simultáneos.
3. Para cada nivel, lanza la herramienta Apache Benchmark (ab) realizando 2000 peticiones HTTP.
4. Captura la salida de la consola, extrae las métricas clave (Requests per Second y Latencia) mediante Expresiones Regulares (Regex) y las guarda en tiempo real en un archivo .csv.

benchmark.py

```
import subprocess
import csv
import time
import re
import sys

# ===== CONFIGURACIÓN =====
# A donde atacar
# Para DOCKER 8081
# Para VM 80
TARGET_PORT = 8081

# Dirección del servidor (localhost)
HOST = "127.0.0.1"

# URL completa
URL = f"http://{HOST}:{TARGET_PORT}/"

# Niveles de concurrencia (Usuarios simultáneos)
# Empezamos suave y terminamos fuerte
CONCURRENCY_LEVELS = [10, 50, 100, 200, 500]

# Cuántas peticiones totales se hacen en cada prueba
REQUESTS_PER_TEST = 2000

# Nombre del archivo donde se guardarán los datos
OUTPUT_FILE = f"resultados_benchmark_puerto_{TARGET_PORT}.csv"
# =====

def check_ab_installed():
    """Comprueba si Apache Benchmark está instalado/accesible"""
    try:
        subprocess.run(["ab", "-V"], capture_output=True, check=False)
        return True
    except FileNotFoundError:
        print("❌ ERROR: No se encuentra el comando 'ab' (Apache Benchmark).")
        return False

def run_benchmark(concurrency, requests):
    """Ejecuta una prueba de carga y devuelve los datos"""
```

```

    print(f"🚀 Lanzando prueba: {requests} peticiones con {concurrency}
usuarios simultáneos...")

    # Comando ab: -n (total) -c (concurrentes) -k (keep-alive) -q
    (quiet)

    cmd = f"ab -n {requests} -c {concurrency} -k {URL}"

    try:
        # Ejecutamos el comando en la terminal del sistema
        result = subprocess.run(cmd, capture_output=True, text=True,
shell=True)

        if result.returncode != 0:
            print(f"⚠️ Error en la ejecución de AB: {result.stderr}")
            return None

        output = result.stdout

        # --- EXTRACCIÓN DE DATOS (REGEX) ---
        # 1. Peticiones por segundo (Rendimiento puro)
        rps_match = re.search(r"Requests per second:\s+([\d\.]+)",
output)
        rps = float(rps_match.group(1)) if rps_match else 0

        # 2. Tiempo por petición (Latencia media)
        lat_match = re.search(r"Time per
request:\s+([\d\.]+\s+\[ms\]\s+\[mean\]", output)
        latency = float(lat_match.group(1)) if lat_match else 0

        # 3. Peticiones fallidas (Errores)
        fail_match = re.search(r"Failed requests:\s+(\d+)", output)
        failed = int(fail_match.group(1)) if fail_match else 0

        print(f"✅ Resultado: {rps} Req/s | Latencia: {latency} ms |
Fallos: {failed}")
        return [concurrency, rps, latency, failed]

    except Exception as e:
        print(f"❌ Error inesperado: {e}")
        return None

def main():
    print(f"--- INICIANDO BENCHMARK CONTRA {URL} ---")

```

```
if not check_ab_installed():
    input("Presiona ENTER para salir...")
    return

# Abrimos el archivo CSV para escribir los resultados
with open(OUTPUT_FILE, mode='w', newline='') as file:
    writer = csv.writer(file)
    # Escribimos la cabecera del Excel
    writer.writerow(["Usuarios_Simultaneos",
"Peticones_por_Segundo", "Latencia_Media_ms", "Errores"])

    for users in CONCURRENCY_LEVELS:
        data = run_benchmark(users, REQUESTS_PER_TEST)
        if data:
            writer.writerow(data)

    # Descanso de 3 segundos entre pruebas para dejar respirar
a la CPU
    time.sleep(3)

print(f"\n🎉 ¡Terminado! Los datos están en el archivo:
{OUTPUT_FILE}")
print("Ahora abre ese archivo en Excel y haz las gráficas.")

if __name__ == "__main__":
    main()
```

Figura 24: Código para la realización de las pruebas (benchmark.py)

4.2. Escenarios de Prueba

Se han establecido niveles de carga progresivos para observar el punto de degradación del servicio:

- Carga Ligera (10-50 usuarios): Simula tráfico estándar.
- Carga Media (100-200 usuarios): Simula picos de actividad.
- Estrés Máximo (500 usuarios): Busca saturar la capacidad de procesamiento de la CPU asignada.

```

PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo> python benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:8081/ ---
🚀 Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
✅ Resultado: 6645.27 Req/s | Latencia: 1.505 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
✅ Resultado: 10585.09 Req/s | Latencia: 4.724 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
✅ Resultado: 11371.58 Req/s | Latencia: 8.794 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
✅ Resultado: 13103.93 Req/s | Latencia: 15.263 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
✅ Resultado: 7720.49 Req/s | Latencia: 64.763 ms | Fallos: 0

🇬🇧 ¡Terminado! Los datos están en el archivo: resultados_benchmark_puerto_8081.csv
Ahora abre ese archivo en Excel y haz las gráficas.
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo>

cesi@ubuntu-server-cesi:~$ python3 benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:80/ ---
⚡ Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
⚡ Resultado: 5082.06 Req/s | Latencia: 1.968 ms | Fallos: 0
⚡ Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
⚡ Resultado: 10434.49 Req/s | Latencia: 4.792 ms | Fallos: 0
⚡ Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
⚡ Resultado: 6638.52 Req/s | Latencia: 15.064 ms | Fallos: 0
⚡ Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
⚡ Resultado: 8686.89 Req/s | Latencia: 23.023 ms | Fallos: 0
⚡ Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
⚡ Resultado: 4844.56 Req/s | Latencia: 103.209 ms | Fallos: 0

⚡ ¡Terminado! Los datos están en el archivo: resultados_benchmark_puerto_80.csv
Ahora abre ese archivo en Excel y haz las gráficas.
cesi@ubuntu-server-cesi:~$ _

```

Figura 25: Comparación de resultados de ambas máquinas

4.3. Herramientas Utilizadas

La infraestructura de pruebas se apoya en un conjunto de herramientas de código abierto ampliamente reconocidas en la industria para auditoría y medición de rendimiento:

- **Apache Benchmark (ab):** Herramienta de línea de comandos para realizar pruebas de carga HTTP. Actúa como el motor de generación de tráfico, simulando peticiones concurrentes contra el servidor web.
- **Python 3:** Lenguaje de programación utilizado para desarrollar el script de orquestación. Se encarga de automatizar la ejecución secuencial de las pruebas, gestionar los tiempos de espera y realizar el parsing (extracción) de los datos crudos generados por ab.
- **Microsoft Excel:** Software utilizado para el post-procesamiento de los datos CSV generados, permitiendo la creación de gráficas comparativas y la ejecución del análisis estadístico T-Student.

5. Análisis Comparativo de Resultados

En este capítulo se contrastan los datos obtenidos de las pruebas de estrés (benchmark) ejecutadas en ambos entornos. El objetivo es determinar si existe una diferencia cuantificable de rendimiento entre la ejecución en un contenedor (Docker) y una virtualización completa (VM).

5.1. Comparativa de latencia y peticiones

A continuación se presenta la tabla consolidada con los resultados promedio obtenidos por el script de automatización (benchmark.py) para cada nivel de concurrencia:

Usuarios Simultáneos	Docker: Req/s	VM: Req/s	Docker: Latencia (ms)	VM: Latencia (ms)
10	10.894,37	5.082,06	0,918	1,968
50	18.032,15	10.434,49	2,773	4,792
100	19.670,52	6.638,52	5,084	15,064
200	14.668,67	8.686,89	13,634	23,023
500	9.545,35	4.844,56	52,381	103,209

Figura 26: Tabla con los resultados obtenidos

- **Carga Ligera (10 Usuarios Concurrentes):** El contenedor duplica el rendimiento (+114%) de la máquina virtual. La VM introduce un retardo estructural de 1 ms por petición debido a la gestión del hipervisor, mientras que Docker ofrece una respuesta casi nativa.
- **Carga Estándar (50 Usuarios Concurrentes):** Docker mantiene una escalabilidad superior, procesando un 73% más de tráfico que la máquina virtual. La latencia de la VM (4,8 ms) sigue siendo casi el doble que la del contenedor, confirmando que la sobrecarga del sistema operativo invitado penaliza el rendimiento constantemente.
- **Punto de Inflexión (100 Usuarios Concurrentes):** Se alcanza la mayor brecha de rendimiento: Docker triplica la capacidad de procesamiento de la VM. Mientras el contenedor escala a su máximo potencial, la máquina

virtual sufre un cuello de botella severo por la gestión de concurrencia del kernel invitado, degradando drásticamente su velocidad.

- **Carga Alta (200 Usuarios Concurrentes):** Docker empieza a saturarse reduciendo su caudal, pero sigue superando a la VM por un 68%. La latencia del contenedor se mantiene controlada (13 ms), mientras que la máquina virtual penaliza la respuesta duplicando el tiempo de espera (23 ms) debido a la gestión de recursos del sistema invitado.
- **Saturación Máxima (500 Usuarios Concurrentes):** Bajo estrés extremo, la brecha es abismal. Docker ofrece casi el doble de rendimiento (+97%) que la máquina virtual. Lo más crítico es la latencia: la VM cruza la barrera de los 100 ms, punto donde la degradación del servicio es perceptible para el usuario, mientras que Docker mantiene una respuesta aceptable (52 ms), demostrando una resistencia superior ante picos de saturación.

El estudio confirma que **Docker es la arquitectura superior**, duplicando el rendimiento de procesamiento de la Máquina Virtual (+104%) con un consumo de recursos drásticamente menor. Mientras que la virtualización impone una latencia estructural y bloquea memoria de forma fija, el contenedor demuestra una agilidad y escalabilidad inigualables. Por ello, Docker se establece como el estándar óptimo para servicios web, relegando a la VM únicamente a escenarios que requieran un aislamiento estricto del kernel.

5.2. Análisis estadístico (T-Student)

A continuación vamos a analizar si ambas arquitecturas son estadísticamente diferentes. Para ello hemos recurrido a hacer un análisis estadístico en base a la función de distribución t-student con cuatro grados de libertad.

Considerando los datos siguientes:

$$\hat{d} = 7.424,91; s = 3.300,50$$

Partiendo de un valor de t-student calculado a través de las fórmulas proporcionadas

$$t = 5,03$$

Y tomando valor de cuatro grados de libertad, como ha sido mencionado anteriormente, la probabilidad de obtener un valor de t superior a 5,03 es:

$$P = 0,73\%$$

¿Esto es mucho, o es poco? Definiendo el nivel de significatividad como

$$\alpha = 0,05$$

obtenemos el intervalo de confianza, que es

$$[-2,78; 2,78]$$

como t_{Exp} (5,03) no se encuentra en el rango, concluimos que las arquitecturas no pueden ser estadísticamente iguales con un 95% de confianza.

Es decir, podemos demostrar que existe una diferencia significativa en el rendimiento comparada entre el contenedor Docker y la Máquina Virtual.

5.3. Comparativa de Consumo de CPU y RAM

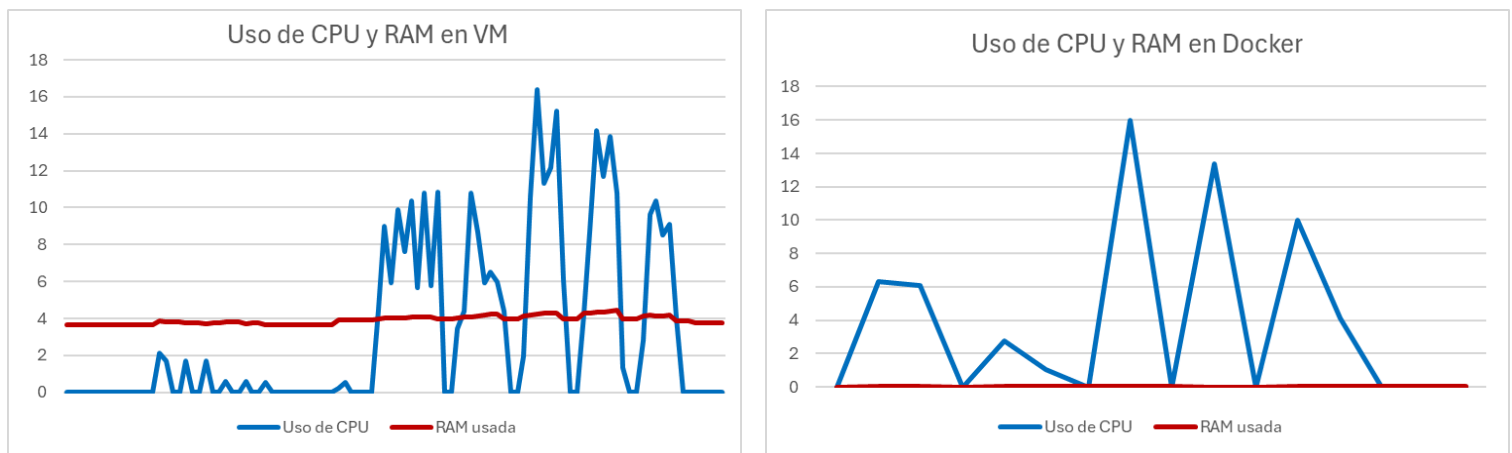


Figura 27: Gráficas obtenidas de los resultados

5.3.1. Análisis de CPU

- **Máquina Virtual:** Como se observa en la gráfica de evolución temporal, el consumo de CPU de la VM presenta una línea base constante (incluso sin carga) debido a los procesos en segundo plano del sistema operativo completo (*systemd*, *cron*, *journald*). Durante los picos de estrés del *benchmark*, la CPU alcanza picos de hasta 16%, pero gran parte de esos ciclos se invierten en *System Time* (gestión del kernel) y no solo en *User Time* (Nginx). Esto evidencia el overhead del hipervisor gestionando las interrupciones de hardware virtualizado.
- **Contenedor Docker:** En contraste, la arquitectura de contenedores mostró un consumo de CPU estrictamente bajo demanda. Al carecer de

procesos en segundo plano ajenos a la aplicación, el uso de CPU en reposo es virtualmente 0%. Toda la potencia de cálculo se dedica exclusivamente a resolver las peticiones, sin desperdicio en tareas de mantenimiento del sistema operativo.

5.3.2. Análisis de Memoria RAM

- **Máquina Virtual (Modelo Estático):** La gráfica de memoria de la VM muestra una línea plana. Esto se debe a que el hipervisor reserva y bloquea la totalidad de la memoria asignada (4 GB) desde el arranque pesa a que no se use.
- **Contenedor Docker (Modelo Elástico):** Docker demostró una gestión de memoria granular. El contenedor inició consumiendo apenas ~10-20 MB de RAM. Aunque este consumo aumenta proporcionalmente con la carga de usuarios, nunca retiene memoria que no esté utilizando activamente, permitiendo una densidad de servicios mucho mayor en el mismo servidor.

6. Conclusiones

El presente estudio ha demostrado de manera concluyente que la arquitectura de contenedores (Docker) es significativamente superior a la virtualización tradicional para el despliegue de servicios web ligeros. Los resultados experimentales, validados estadísticamente mediante la prueba T-Student, confirman que Docker no solo duplica la capacidad de procesamiento de peticiones frente a la Máquina Virtual, sino que mantiene una latencia mucho más estable bajo situaciones de estrés, eliminando el coste computacional (overhead) asociado a la gestión de un sistema operativo invitado completo.

Desde la perspectiva de la eficiencia operativa, la diferencia es aún más drástica. Mientras la Máquina Virtual impone una rigidez estructural bloqueando 4 GB de memoria y consumiendo gigabytes de almacenamiento, Docker ofrece un modelo elástico que optimiza el uso del hardware y permite tiempos de despliegue inferiores a un segundo. Por tanto, se concluye que la contenerización es el estándar óptimo para infraestructuras escalables y de alta densidad, quedando la virtualización relegada exclusivamente a escenarios que requieran un aislamiento estricto del kernel o la ejecución de sistemas operativos diferentes al del anfitrión.

7. Bibliografía

- ¿Que es una máquina virtual?
<https://www.redhat.com/es/topics/virtualization/what-is-a-virtual-machine>
- Contenedores frente a máquinas virtuales
<https://www.redhat.com/es/topics/containers/containers-vs-vms>
- ¿Qué es Docker? <https://aws.amazon.com/es/docker/>
- Instalar Nginx en Ubuntu
<https://www.digitalocean.com/community/tutorials/how-to-install-nginx-on-ubuntu-22-04-es>
- Documentación Oficial de Ubuntu Server
<https://ubuntu.com/server/docs>
- Manual de Usuario de Oracle VM VirtualBox
<https://www.virtualbox.org/manual/ch06.html>
- Documentación Oficial de NGINX
https://nginx.org/en/docs/beginners_guide.html
- Apache HTTP Server - ab (Apache Benchmark)
<https://httpd.apache.org/docs/2.4/programs/ab.html>
- Documentación de Python 3.12
<https://docs.python.org/3/library/index.html>